# SCSM2223 CROSS-PLATFORM APPLICATION DEVELOPMENT

## MOCK FINAL EXAMINATION — SEMESTER 2, 2025/2026

## ANSWER SCHEME AND EXAMINER NOTES

*For self-marking after attempting the paper under exam conditions (3 hours, closed book).*

## SECTION A — MULTIPLE-CHOICE KEY (20 × 1 MARK)

| No. | Ans | Why |
|---|---|---|
| 1 | **B** | Develop once, run on multiple OS/devices with minimal modification. |
| 2 | **D** | JVM exists per-OS; bytecode is portable — WORA. |
| 3 | **C** | SPA: one initial load, JS updates page sections via API data. |
| 4 | **A** | Chrome–Blink. (Firefox–Gecko, Safari–WebKit, old IE–Trident.) |
| 5 | **C** |  = semantic importance; <b> = visual only. |
| 6 | **B** | Mobile-first adds styles upward with min-width. |
| 7 | **D** | querySelectorAll(".card") returns ALL matches (NodeList). |
| 8 | **A** | Event delegation relies on bubbling; works for future elements. |
| 9 | **C** | Double quotes required; no trailing comma; keys quoted. |
| 10 | **B** | stringify: object → JSON string. parse: string → object. |
| 11 | **B** | 201 Created for successful POST creation. |
| 12 | **A** | response.json() is async and returns another Promise. |
| 13 | **C** | pending → fulfilled / rejected. |
| 14 | **B** | Synchronous lines A then B run first; the fetch callback (C) runs later. |
| 15 | **A** | Vue = progressive JavaScript framework for building UIs. |
| 16 | **D** | v-if adds/removes from DOM; v-show toggles CSS display. |
| 17 | **C** | In script code a ref needs .value; templates auto-unwrap. |
| 18 | **A** | Props down, events up — one-way data flow. |
| 19 | **B** | PUT is idempotent; POST and PATCH are not (PATCH typically not). |
| 20 | **D** | header.payload.signature, base64url-encoded, dot-separated. |

⚠ ***Trap to watch:*** *Q14 is the classic async-order trap — the two console.log lines always print before any .then() callback, even if the network is instant. Q19: memorise the idempotency table (GET/PUT/DELETE idempotent; POST/PATCH not). Q12: the double-.then exists because res.json() itself returns a Promise.*

# SECTION B — MODEL ANSWERS

## QUESTION 1 (8 marks) — Web Application Architecture

**a)** The browser (client) sends an HTTP request to the web server [1]. The server receives and processes the request — it runs the business logic and, where needed, queries the database, which stores and returns the application data [1]. The server then builds an HTTP response containing HTML (or JSON for an API call) and sends it back to the browser [1]. The browser parses and renders the response to display the page; the cycle is called the request–response cycle over the HTTP/HTTPS protocol [1].

*1 mark each: request via HTTP; server processing + DB role; response returned; browser renders. Protocol must be named for full marks.*

**b)** Frontend — the part running in the browser (HTML, CSS, JavaScript) that displays the UI and interacts with the user [1]. Backend — the server-side logic that receives requests, applies business rules, and prepares responses [1]. Database — stores and manages the application's persistent data [1]. API — the defined set of rules/endpoints through which the frontend and other clients communicate with the backend, typically exchanging JSON [1].

⚠ ***Trap to watch:*** *One sentence each is enough — do not waste exam time writing paragraphs here. These are the Dr's exact 'mastery' checklist items.*

## QUESTION 2 (10 marks) — Asynchronous JavaScript

**a)** Asynchronous programming allows long-running operations (such as network requests) to run in the background without blocking the main thread [1]. JavaScript is single-threaded, but fetch() hands the request to the browser and registers a callback; the event loop continues executing other code and runs the callback only when the response arrives [1]. Therefore the page stays responsive — the browser does not freeze while waiting [1].

**b)** Model answer:

```
async function getStudent() {
  try {
    const response = await fetch("https://api.example.com/students/5");
    if (!response.ok) {
      throw new Error(`HTTP Error! Status: ${response.status}`);
    }
    const student = await response.json();
    console.log(student.name);
  } catch (error) {
    console.log("Error:", error.message);
  }
}
```

*Marks: async keyword on function [1]; await fetch [1]; response.ok check that throws Error with status [1]; await response.json() [1]; try/catch wrapping with the error logged [1].*

⚠ ***Trap to watch:*** *Most common losses: forgetting the async keyword, forgetting await on response.json() (it returns a Promise!), and putting the ok-check after .json(). Write string interpolation with backticks or concatenation — both accepted.*

**c)** Advantage: async/await reads like synchronous code, making it easier to read, reason about, and debug (and errors are handled with the familiar try/catch) [1]. Disadvantage: await pauses that

function, so code can accidentally become sequential when requests could run in parallel; it must also be used inside an async function (extra wrapping) [1].

## QUESTION 3 (12 marks) — Vue.js

**a)** A frontend framework is a structured library (e.g., Vue) that lets developers build UIs declaratively — the UI is described as a function of data and the framework updates the DOM automatically [1]. Problems it solves (any two): manual DOM updates for every state change; UI and data getting out of sync; spaghetti code in large apps; no reusable components; hard to test/maintain [2].

**b)** The three blocks: <template> — the HTML-like markup of the component [1]; <script setup> — the component's logic using the Composition API (reactive state, props, functions) [1]; <style scoped> — the component's CSS [1]. The scoped attribute makes the CSS apply ONLY to this component's elements, preventing styles leaking into other components [1].

**c)** ref() wraps the value in a reactive object, so the actual data lives in its .value property — script code must read/write count.value [1]. Inside templates Vue automatically unwraps refs, so {{ count }} is used without .value [1].

**d)** :key gives Vue a stable identity for each rendered item so its diffing algorithm can track which item is which when the list changes [1]. Using the array index breaks this: after a deletion/reorder the indices shift, so Vue may reuse the wrong DOM elements, causing wrong state or lost input [1]. Therefore a unique, stable value (usually the item's id) should be used [1].

⚠ ***Trap to watch:*** *The Dr highlighted ':key is important!' on the slide — near-certain to be tested. Also be ready for the v-if vs v-show and ref() vs reactive() comparisons.*

## QUESTION 4 (15 marks) — Backend, Security and Mobile

**a)** REST (REpresentational State Transfer) is an architectural style where everything is a resource (a noun) with a stable URI, manipulated using standard HTTP verbs, with the server returning representations (typically JSON); requests are stateless [2]. Resources are named with plural nouns, no verbs in the URI (e.g., GET /products/12, not /getProduct?id=12) [1]. Verb mapping: POST = Create, GET = Read, PUT = Update (replace), DELETE = Delete [1].

**b)** 1) The user submits the login form; the frontend POSTs the credentials to the auth endpoint [1]. 2) The server validates the credentials against the database (hashed password check) and, if valid, signs and returns a JWT [1]. 3) The client stores the token (e.g., localStorage / Pinia store) [1]. 4) On every subsequent request the client sends the token in the Authorization: Bearer <token> header [1]. 5) Middleware on the server verifies the token's signature and expiry; if valid it lets the request reach the protected route, which returns the data — if missing/invalid it returns 401 Unauthorized [1].

⚠ ***Trap to watch:*** *401 vs 403 trap: 401 = not authenticated (missing/invalid token); 403 = authenticated but not allowed (e.g., wrong role/ownership). Examiners love this pair.*

**c)** Any two, e.g.: SQL Injection — attacker input is concatenated into an SQL string and changes the query's meaning (e.g., admin' --); prevented with PDO prepared statements (parameterised placeholders) [2]. Cross-Site Scripting (XSS) — malicious script is injected into content rendered for other users; prevented by escaping output with htmlspecialchars() (PHP) or relying on Vue's auto-escaping {{ }} and avoiding v-html [2]. (Also accepted: data/parameter tampering — prevented with server-side validation and JWT signature; replay — short exp TTL.)

**d)** The Vue project is built (npm run build) into static files in dist/, and Capacitor wraps that same dist/ folder inside a native Android project (npx cap add android, npx cap sync) [1]. The app runs inside a WebView — a native shell that renders the web app — while Capacitor's plugin API bridges to native features (camera, GPS); the result opens in Android Studio and builds into an installable APK/AAB [1].

# SECTION C — MODEL CODE ANSWERS

## QUESTION 5 (10 marks) — HTML, CSS and the DOM

**a)** Model answer:

```
<body>
  <header>
    <h1>Student Feedback</h1>
  </header>
  <nav>
    <ul>
      <li><a href="index.html">Home</a></li>
      <li><a href="feedback.html">Feedback</a></li>
    </ul>
  </nav>
  <main>
    <section>
      <h2>Tell us what you think</h2>
      <p>Your feedback helps us improve the course.</p>
    </section>
  </main>
  <footer>
    <address>Faculty of Computing, UTM, 81310 Johor Bahru</address>
  </footer>
</body>
```

*1 mark each: header+h1; nav with two links; main with section (heading+paragraph); footer with address. Semantic tags are required —  everywhere loses the marks.*

**b)** Model answer (mobile-first):

```
.card-grid {
  display: grid;
  grid-template-columns: 1fr;
}

@media (min-width: 1024px) {
  .card-grid {
    grid-template-columns: repeat(3, 1fr);
  }
}
```

*1 mark: base one-column grid; 1 mark: correct min-width media query with repeat(3, 1fr) (or 1fr 1fr 1fr).*

**c)** Model answer:

```
const input = document.getElementById("studentName");
const btn = document.getElementById("addBtn");
const list = document.getElementById("nameList");

btn.addEventListener("click", function () {
  const name = input.value.trim();
  if (name === "") {
    alert("Name cannot be empty!");
    return;
  }
  const li = document.createElement("li");
  li.textContent = name;
```

```
  list.appendChild(li);
  input.value = "";
});
```

*Marks: addEventListener on the button [1]; read + trim + empty check with alert [1]; createElement + textContent [1]; appendChild + clear input [1].*

⚠ ***Trap to watch:*** *The paper explicitly bans innerHTML — using it scores zero for that step. This mirrors the lab constraint ('pure DOM' methods only).*

## QUESTION 6 (10 marks) — Vue Component

Model answer:

```
<template>
  <div>
    <h2>{{ title }}</h2>
    <input v-model="newCourse" placeholder="Course name">
    <button @click="addCourse">Add</button>
    <ul>
      <li v-for="(course, index) in courses" :key="index">
        {{ course }}
      </li>
    </ul>
  </div>
</template>

<script setup>
import { ref } from 'vue'

defineProps({
  title: { type: String, required: true }
})

const newCourse = ref('')
const courses = ref([])

function addCourse() {
  const name = newCourse.value.trim()
  if (name === '') return
  courses.value.push(name)
  newCourse.value = ''
}
</script>
```

*Marks: SFC structure with <template> + <script setup> [2]; defineProps with type+required and {{ title }} in <h2> [2]; ref for newCourse + v-model binding [2]; add function with trim + empty guard + clearing input, wired with @click [2]; v-for with :key rendering [2].*

⚠ ***Trap to watch:*** *Inside the function you MUST write newCourse.value / courses.value — forgetting .value in script is the most common Vue mark-loss. In the template, no .value. (Using each course string itself as :key is also acceptable here; index is tolerated because items are only appended — but if the question says items can be deleted, use a unique id.)*

## QUESTION 7 (15 marks) — Slim 4, PDO and JWT

**a)** Model answer:

```
$app->post('/api/books', function (Request $req, Response $res) {
    $body = (array)$req->getParsedBody();

    if (empty($body['title'])) {
        $res->getBody()->write(json_encode(['error' => 'title is required']));
        return $res
            ->withHeader('Content-Type', 'application/json')
            ->withStatus(400);
    }

    $res->getBody()->write(json_encode(['message' => 'Book created']));
    return $res
        ->withHeader('Content-Type', 'application/json')
        ->withStatus(201);
});
```

*Marks: correct $app->post signature with Request/Response [1]; getParsedBody read [1]; validation branch returning 400 with error JSON [1]; success branch 201 with message JSON [1]; json_encode + write + withHeader + RETURNING the response [1].*

⚠ ***Trap to watch:*** *PSR-7 responses are IMMUTABLE — withHeader()/withStatus() return a NEW object, so you must return the chained result. Forgetting the return (or echoing instead of writing to the body) is the classic Slim mark-killer. The Dr's slides repeat this three times.*

**b)** Model answer:

```
$sql = 'INSERT INTO books (title, author, year)
        VALUES (:title, :author, :year)';
$stmt = $pdo->prepare($sql);
$stmt->execute([
    ':title'  => $title,
    ':author' => $author,
    ':year'   => $year,
]);
$newId = (int)$pdo->lastInsertId();
```

*Marks: correct INSERT with named placeholders [2]; prepare() [1]; execute() with the value array [1]; lastInsertId() into $newId [1].*

⚠ ***Trap to watch:*** *The course teaches prepare() + execute([...]) — bindParam() never appears in the slides. Do NOT reuse the same named placeholder twice in one query (HY093 error when emulation is off). Values must never be concatenated into the SQL string.*

**c) i)** SQL Injection. Example malicious input: `1 OR 1=1` (returns every row) or `1; DROP TABLE books;--` [1].

**ii)** Model answer:

```
$id = $_GET['id'];
$stmt = $pdo->prepare('SELECT * FROM books WHERE id = :id');
$stmt->execute([':id' => $id]);
$rows = $stmt->fetchAll();
```

*2 marks: prepare with placeholder [1]; execute with the bound value + fetchAll from the statement (not from query()) [1]. Positional ? placeholder equally accepted.*

**d)** Header: `Authorization: Bearer <token>` [1]. The middleware verifies the token's SIGNATURE (signed with the server's secret key — detects tampering) and its EXPIRY (the exp claim) before allowing access [1].

## HOW THIS PAPER MAPS TO THE DR'S TIPS

**Cognitive split (as per the Dr's tips document):** Remember ≈ 20 marks (Section A recall items), Understand ≈ 45 marks (Section B explanations/comparisons), Apply ≈ 35 marks (Section C code writing + Q2(b) conversion). Two-thirds of the paper rewards explaining and applying — exactly the shape the Dr described.

**Topic weights:** Backend stack (REST/Slim/PDO/JWT/security/Capacitor) ≈ 33 marks — the largest block, as the tips promise. Vue.js ≈ 26 marks (structured Q3 + code Q6 + four MCQs). Asynchronous JavaScript ≈ 13 marks (MCQ + the .then → async/await conversion, the tips' named skill). JavaScript & DOM and Web architecture & HTML/CSS make up the remaining fast-marks questions.

**Training advice:** attempt the full booklet in one 3-hour closed-book sitting, writing code by hand. Then mark yourself with this scheme. Anything below full marks on Q2(b), Q6, Q7(a) or Q7(b) — retype those answers at the keyboard until they run. Those four items are the Dr's explicitly listed mastery skills and the likeliest real-exam questions.